Practical Lab: Access Control Models

DAC, MAC, RBAC and ABAC

Zineb Elansari
UM6P College of Computing, Ben Guerir, Morocco

November 10, 2026

Contents

1 Introduction and Environment **2**

2 Discretionary Access Control (DAC) **2**

2.1 Guided Experiment 2

2.2 Challenge 1: Confidential Documents 3

3 Mandatory Access Control (MAC) **4**

3.1 Bell-LaPadula Model 4

3.2 SELinux Exploration 4

3.3 Challenge 2: Military Information System 4

4 Role-Based Access Control (RBAC) **5**

4.1 Guided Experiment 5

4.2 Challenge 3: Enterprise Access Management 6

5 Attribute-Based Access Control (ABAC) **9**

5.1 Challenge 4: Banking Access Policy 9

6 Conclusion **9**

1 Introduction and Environment

This report presents the experiments of the practical lab on access control models: Discretionary Access Control (DAC), Mandatory Access Control (MAC), Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC).

The experiments were carried out on Ubuntu 24.04.5 LTS running under WSL 2 on Windows 11. The Linux account `zinouba` was used, with `sudo` for administrative operations. Figure 1 shows the verification of the environment.



Figure 1: Figure 1 — Linux environment verification.

2 Discretionary Access Control (DAC)

2.1 Guided Experiment

In DAC, the owner of a resource determines who can access it. A file in Linux has an owner, a group and permissions for others, each combining read (4), write (2) and execute (1).

A working directory and a file `report.txt` were created, and the permissions were set to `740`: the owner can read, write and execute, the group can only read, and others have no access (Figure 2.1).

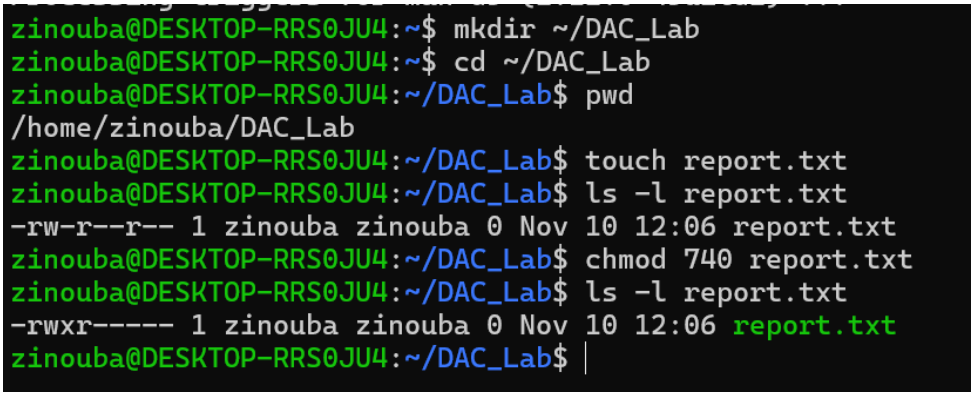


Figure 2: Figure 2 — Setting DAC permissions using chmod 740.

The group `accounting` was then created and the ownership of the file was changed to `root:accounting`. The permissions remained `-rwxr---`, while the owner and group changed (Figure 2.1).

```
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo groupadd accounting
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo chown root:accounting re
port.txt
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ ls -l report.txt
-rwxr----- 1 root accounting 0 Nov 10 12:06 report.txt
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$
```

Figure 3: Figure 3 — Changing file ownership to root and the accounting group.

2.2 Challenge 1: Confidential Documents

Policy. The owner can read and modify documents, the accounting group can only read them, and other users have no access.

Configuration.

```
sudo mkdir -p /company/finance
echo "Confidential financial report - 2026" | sudo tee /company/finance/
    financial.txt
sudo chown root:accounting /company/finance
sudo chown root:accounting /company/finance/financial.txt
sudo chmod 750 /company/finance
sudo chmod 640 /company/finance/financial.txt
```

The directory uses `750` (owner: full access, group: read and traverse, others: none) and the file uses `640` (owner: read and write, group: read, others: none).

Verification. Figure 2.2 shows the resulting ownership and permissions: `drwxr-x--` on the directory and `-rw-r---` on the file, both owned by `root:accounting`. The first `ls -l` on the file, executed as the ordinary user `zinouba`, returns *Permission denied*. This is consistent with the policy, since `zinouba` is neither the owner nor a member of `accounting`. The same command run with `sudo` displays the file.

```
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo mkdir -p /company/finance
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ echo "Confidential financial report - 2026" | sudo tee /company/finance/financial.txt
Confidential financial report - 2026
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo cat /company/finance/financial.txt
Confidential financial report - 2026
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo chown root:accounting /company/finance
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo chown root:accounting /company/finance/financial.txt
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo chmod 750 /company/finance
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo chmod 640 /company/finance/financial.txt
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ ls -ld /company/finance
drwxr-x--- 2 root accounting 4096 Nov 10 12:07 /company/finance
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ ls -l /company/finance/financial.txt
ls: cannot access '/company/finance/financial.txt': Permission denied
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo ls -l /company/finance/financial.txt
-rw-r----- 1 root accounting 37 Nov 10 12:07 /company/finance/financial.txt
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$
```

Figure 4: Figure 4 — Ownership and permissions enforcing the company's financial-document policy.

Access tests. Two test users were created: `accountant1`, added to the `accounting` group, and `outsider1`, who belongs to no relevant group. As shown in Figure 2.2, `accountant1` reads the document successfully, whereas `outsider1` receives *Permission denied*.

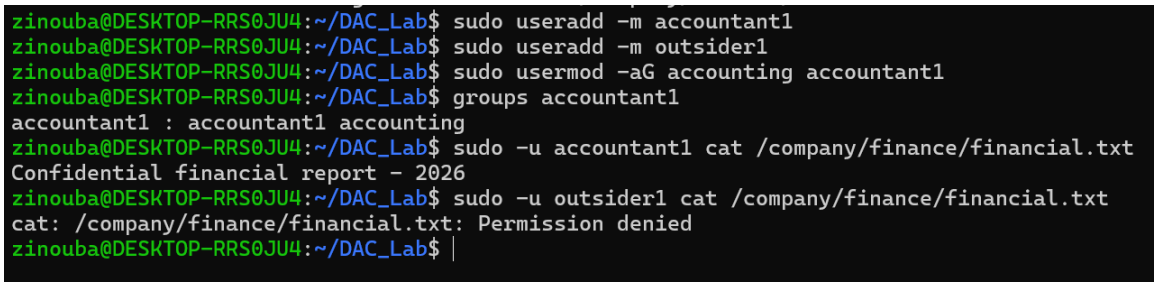


Figure 5: Figure 5 — Authorized accounting-group access and denied unauthorized access.

Question 6. *The directory has permission 750, but a file inside has permission 600. Can a member of the accounting group read the file?*

No. The permission `750` allows members of `accounting` to traverse the directory, but the file permission `600` grants read and write access exclusively to its owner. A group member who is not the owner therefore cannot read the file. Both the directory traversal permission and the file read permission must be satisfied.

3 Mandatory Access Control (MAC)

3.1 Bell-LaPadula Model

In MAC, a central security policy determines which operations are allowed, and resource owners cannot freely override it. The Bell-LaPadula model preserves confidentiality with two rules:

- **No Read Up**: a user cannot read a document classified above their clearance.
- **No Write Down**: a user cannot write information into a document of a lower classification.

The classification levels are Public (1), Confidential (2) and Secret (3).

3.2 SELinux Exploration

SELinux was investigated with `sestatus` and `ls -Z` (Figure 3.2). The command `sestatus` reports *SELinux status: disabled*, and `ls -Z` displays `?` as the security context of `report.txt`.

The Ubuntu WSL 2 environment therefore does not provide active SELinux enforcement or usable SELinux file contexts. Consequently, the MAC model is analyzed below using the Bell-LaPadula rules, and no kernel-level SELinux enforcement is claimed.

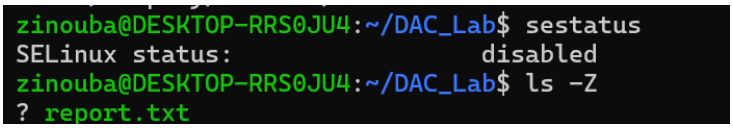


Figure 6: Figure 6 — Inspecting SELinux availability and security-label support under Ubuntu WSL 2.

3.3 Challenge 2: Military Information System

Question 1. Read-access matrix (No Read Up).

User (clearance)	Document_A (Secret)	Document_B (Confidential)	Document_C (Public)
Alice (Secret)	Allow	Allow	Allow
Bob (Confidential)	Deny	Allow	Allow
Charlie (Public)	Deny	Deny	Allow

Question 2. Bob reads Document_A. Access is **denied**. Bob has Confidential clearance and Document_A is Secret, which is above his clearance. The operation violates No Read Up.

Question 3. Alice writes into Document_C. The operation is **denied**. Alice has Secret clearance and Document_C is Public. Writing into a lower classification violates No Write Down.

Question 4. Preventing information leakage. Bell-LaPadula controls information flow with mandatory classification levels. No Read Up prevents users with a lower clearance from reading sensitive documents, while No Write Down prevents users with a higher clearance from copying sensitive information into lower-classified resources.

Question 5. MAC versus DAC. Access is not necessarily granted. When a system enforces both DAC and MAC, both policies must allow the operation. A user with sufficient clearance can still be refused by the ordinary Linux permissions set by the owner. The difference is that DAC is decided by the owner of the resource, whereas MAC is decided by a central policy that the owner cannot change.

4 Role-Based Access Control (RBAC)

4.1 Guided Experiment

RBAC assigns permissions to roles rather than to individual users. The roles were simulated with Linux groups: `administrators`, `developers` and `viewers`. Three users were created and assigned as follows: Alice (Developer), Bob (Viewer) and Charlie (Administrator). Figure 4.1 shows the group memberships.

```
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo groupadd administrators
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo groupadd developers
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo groupadd viewers
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo useradd -m alice
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo useradd -m bob
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo useradd -m charlie
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo usermod -aG developers alice
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo usermod -aG viewers bob
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo usermod -aG administrators charlie
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ groups alice
alice : alice developers
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ groups bob
bob : bob viewers
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ groups charlie
charlie : charlie administrators
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$
```

Figure 7: Figure 7 — Assigning employees to the developers, viewers and administrators roles.

The directories `/company/source` and `/company/reports` were created with the following initial configuration (Figure 4.1):

- `/company/source`: owner `root`, group `developers`, mode `770`;

- `/company/reports`: owner `root`, group `viewers`, mode `750`.

```
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo mkdir -p /company/source
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo mkdir -p /company/reports
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo chown root:developers /company/source
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo chmod 770 /company/source
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo chown root:viewers /company/reports
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo chmod 750 /company/reports
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ ls -ld /company/source /company/reports
drwxr-x--- 2 root viewers    4096 Nov 10 12:23 /company/reports
drwxrwx--- 2 root developers 4096 Nov 10 12:23 /company/source
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$
```

Figure 8: Figure 8 — Initial permissions configured for source-code and report directories.

4.2 Challenge 3: Enterprise Access Management

Policy.

Role	Source code	Reports
Developer	Read / Write	Read
Viewer	No access	Read
Administrator	Full access	Full access

Does the initial configuration satisfy the policy? No. Two requirements are not met:

1. Alice (Developer) cannot access `/company/reports`, although developers must have read access.
2. Charlie (Administrator) cannot access either directory, because Linux does not grant special rights to a group merely because of its name.

Figure 4.2 confirms both denials.

```
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo -u alice ls /company/reports
ls: cannot open directory '/company/reports': Permission denied
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo -u charlie ls /company/source
ls: cannot open directory '/company/source': Permission denied
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$
```

Figure 9: Figure 9 — Missing permissions for developers and administrators in the initial RBAC configuration.

Correction with POSIX ACLs. Standard Unix permissions provide a single owning group, whereas the policy requires several groups with different rights on the same resource. POSIX ACLs were therefore used:

```
sudo setfacl -m g:administrators:rwx /company/source
sudo setfacl -m g:developers:rx      /company/reports
sudo setfacl -m g:administrators:rwx /company/reports
```

Sample files were then created and configured:

```
echo "Company source code" | sudo tee /company/source/app.txt
echo "Quarterly company report" | sudo tee /company/reports/report.txt
sudo chown root:developers /company/source/app.txt
sudo chmod 660 /company/source/app.txt
sudo chown root:viewers /company/reports/report.txt
sudo chmod 640 /company/reports/report.txt
sudo setfacl -m g:administrators:rw  /company/source/app.txt
sudo setfacl -m g:developers:r--     /company/reports/report.txt
sudo setfacl -m g:administrators:rw- /company/reports/report.txt
```

```
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo setfacl -m g:administrators:rwx /company/source
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo setfacl -m g:developers:rx /company/reports
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo setfacl -m g:administrators:rwx /company/reports
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ echo "Company source code" | sudo tee /company/source/app.txt
Company source code
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ echo "Quarterly company report" | sudo tee /company/reports/report.txt
Quarterly company report
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo chown root:developers /company/source/app.txt
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo chmod 660 /company/source/app.txt
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo chown root:viewers /company/reports/report.txt
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo chmod 640 /company/reports/report.txt
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo setfacl -m g:administrators:rw /company/source/app.txt
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo setfacl -m g:developers:r-- /company/reports/report.txt
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ sudo setfacl -m g:administrators:rw- /company/reports/report.txt
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ getfacl /company/source
```

Figure 10: Figure 10a — ACL configuration commands.

The resulting ACLs are shown in Figures 10b and 10c. On the directories, `/company/source` contains `group:administrators:rwx`, and `/company/reports` contains `group:administrators:rwx` and `group:developers:r-x`. On the files, `app.txt` contains `group:administrators:rw-`, and `report.txt` contains `group:administrators:rw-` and `group:developers:r-`. In every case the mask allows the granted rights. Reading the ACL of `app.txt` required `sudo`, because the ordinary user `zinouba` has no access to that directory.

```
zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ getfacl /company/source
getfacl: Removing leading '/' from absolute path names
# file: company/source
# owner: root
# group: developers
user::rwx
group::rwx
group:administrators:rwx
mask::rwx
other::---

zinouba@DESKTOP-RRS0JU4:~/DAC_Lab$ getfacl /company/reports
getfacl: Removing leading '/' from absolute path names
# file: company/reports
# owner: root
# group: viewers
user::rwx
group::r-x
group:administrators:rwx
group:developers:r-x
mask::rwx
other::---
```

Figure 11: Figure 10b — Extended ACLs of the source and reports directories.

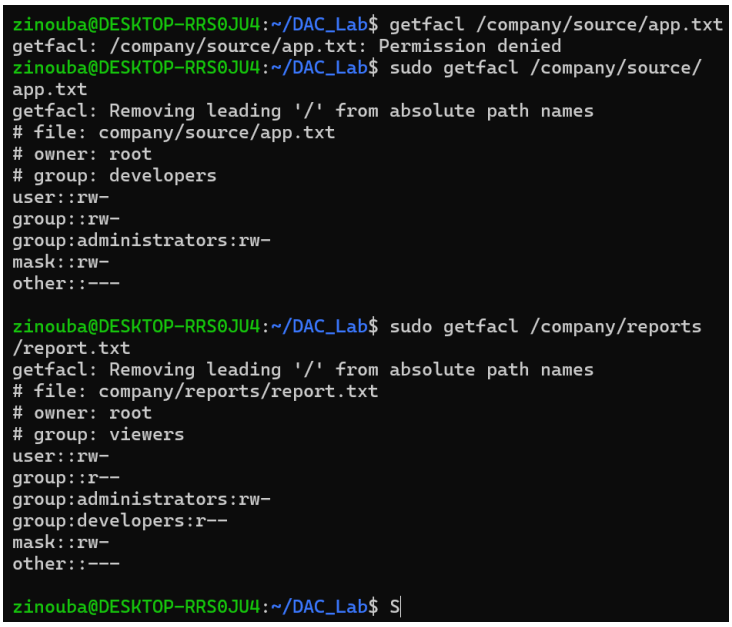


Figure 12: Figure 10c — Extended ACLs of the sample source and report files.

Verification of the three roles. Figure 4.2 shows the tests executed with `sudo -u`. The observed results match the policy:

User (role)	Source read	Source write	Report read	Report write
Alice (Developer)	Allow	Allow	Allow	Deny
Bob (Viewer)	Deny	Deny	Allow	Deny
Charlie (Administrator)	Allow	Allow	Allow	Allow

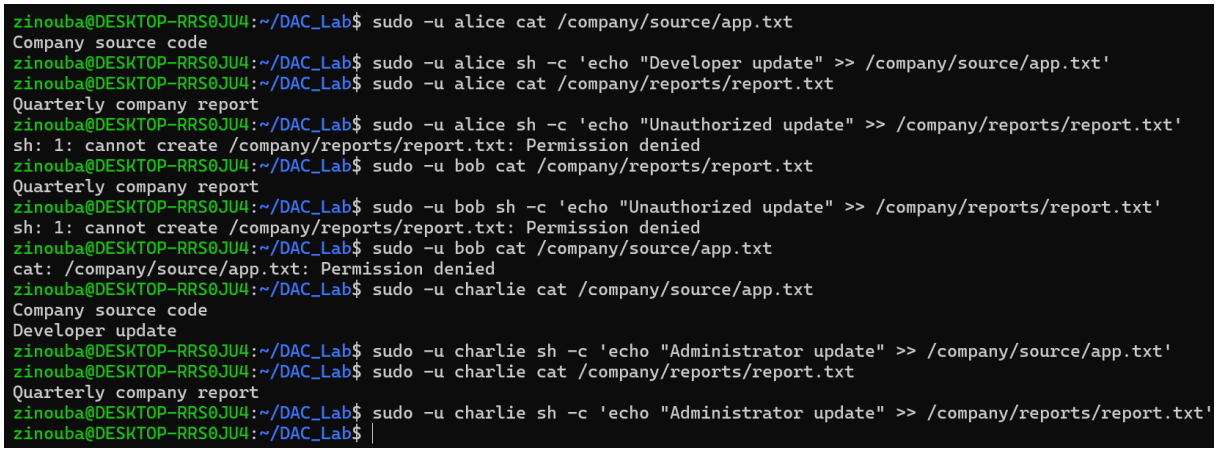


Figure 13: Figure 11 — Validating role-based permissions using three distinct Linux users.

Note. This solution configures the existing directories and sample files. Newly created files do not automatically receive the same ACLs. In a production system, default ACL inheritance or a provisioning policy would maintain consistent permissions.

5 Attribute-Based Access Control (ABAC)

ABAC bases authorization on attributes of the subject (department, clearance), the object (department, classification), the action and the environment (time). In Challenge 4, the bank grants access only if all three conditions hold: the employee and the document belong to the same department, the clearance is sufficient, and the request is made between 08:00 and 18:00.

5.1 Challenge 4: Banking Access Policy

Questions 1 and 2. Decisions and responsible attribute.

User	Department	Clearance	Document	Time	Decision
Alice	Finance	Secret	Finance / Secret	10:00	**GRANT**
Bob	HR	Confidential	Finance / Confidential	11:00	**DENY** (department)
Charlie	Finance	Public	Finance / Secret	14:00	**DENY** (clearance)
David	Finance	Secret	Finance / Secret	22:00	**DENY** (time)

Question 3. Alice requests an HR document: access is **denied**, because Alice belongs to Finance and the department condition fails.

Question 4. David requests access at 17:00: the decision changes to **grant**. His department matches, his clearance is sufficient and 17:00 lies within the permitted period.

Question 5. ABAC is more flexible than RBAC because it evaluates combinations of subject, resource, action and environmental attributes. It can grant or deny access dynamically according to context, such as the time of day, without changing the user's role.

6 Conclusion

The lab illustrated four access control models. DAC was implemented with Linux ownership and permissions, and verified with authorized and unauthorized users. MAC was studied through the Bell-LaPadula rules, since SELinux is disabled in the WSL 2 environment. RBAC was implemented with Linux groups completed by POSIX ACLs, which allowed the full role policy to be satisfied. ABAC was evaluated on the banking policy, showing how contextual attributes such as time refine access decisions.